

Eksperyment 4.3 – Bootstrapped Fine-Tuning

Klasyczny, deterministyczny klasyfikator jako darmowy "nauczyciel" – czy model LLM douczony na jego etykietach zinternalizuje regułę rozpoznawania flag bitowych SAM, bez potrzeby zewnętrznej korekty w czasie działania?

STATUS: PROPOZYCJA – ETAP A UKOŃCZONY, ETAPY B/C/D NIEWYKONANE

1. GENEZA – SUGESTIA WYKŁADOWCY

„Ewentualnie można zrobić bazę wzorców i na jej podstawie nauczyć LLM. Czyli najpierw LLM wynajduje wzorce. A potem douczamy inny lub ten sam LLM wyposażony w plik z listą wzorców/reguł.”

– ODPOWIEDŹ WYKŁADOWCY NA WYNIK EKSPERYMENTU 4.1 (HYBRYDOWY OVERRIDE)

2. KLUCZOWA KOREKTA POMYSŁU

✗ Pierwotny pomysł

LLM sam "wynajduje wzorce" (odkrywa regułę z przykładów) – trudne, kosztowne, niepewne zadanie samo w sobie.

✓ Nasza propozycja

"Nauczycielem" nie musi być LLM. W Eksperymentcie 4.1 klasyczny, deterministyczny klasyfikator (Kierunek B) rozpoznaje wzorzec flag bitowych ze 100% skutecznością, za darmo, bez API. Użyjmy GO jako automatycznego etykiciarza – to klasyczna **destylacja wiedzy z modelu symbolicznego do sieci neuronowej**.

3. PRAWDZIWA PRZESZKODA: NIE BRAK ALGORYTMU, TYLKO BRAK RÓŻNORODNOŚCI DANYCH

RYZIKO PRZEUCZENIA

Oryginalny mini-DBC z Eksperymentu 4.1 ma tylko **3 CAN ID i JEDEN** przypadek flag

bitowych. Fine-tuning na tak wąskim zbiorze nauczyłby model rozpoznawać te KONKRETNE 3 ramki, nie ogólną zasadę "mały, nie-monotoniczny zbiór wartości = flagi bitowe".

- flagi bitowe w różnych pozycjach bajtu (nie tylko bajt 0)
- różna liczba flag w bajcie (2-6)
- flagi rozproszone na wielu bajtach tej samej ramki
- bajty mieszane (flaga + częściowy skalar w tym samym bajcie)
- skalary o różnych zakresach/dynamikach zmian

4. PROPONOWANY PIPELINE – 4 ETAPY

ETAP A Rozbudowa generatora 40 zróżnicowanych konfiguracji CAN ID, 5 wzorców sygnałów (<code>generate_traffic_diverse.py</code>). ✓ UKOŃCZONE I ZWERYFIKOWANE	ETAP B Auto-etykietowanie Klasyfikator Kierunku B (<code>looksLikeBitFlags()</code>) etykietuje korpus automatycznie, bez człowieka. NIEWYKONANE	ETAP C Format treningowy Etykietowane próbki → pary (prompt, completion) w schemacie identycznym z <code>DecodingAccuracyRunner</code>. NIEWYKONANE	ETAP D Fine-tuning + ewaluacja Douczenie GPT-4.1/o4-mini lub DeepSeek, ocena tym samym harnessem co Eksperyment 4.1 (N=100, zero-shot, bez override'u). NIEWYKONANE
--	---	--	--

5. KRYTERIA SUKCESU I PORAŻKI

Sukces Douczoney model osiąga na sygnałach dyskretnych detection rate zbliżony do wyniku z override'em (~94%) – SAMODZIELNIE, bez zewnętrznej klasycznej łatki w czasie działania.	Porażka (też wartościowa!) Douczoney model nie generalizuje poza konfiguracje widziane podczas treningu – sygnał, że korpus wciąż za wąski ALBO problem jest głębiej osadzony w sposobie reprezentacji danych liczbowych przez model.
--	--

Koszt/zakres: znacząco większy niż dotychczasowe eksperymenty tej serii – nowy, nietrywialny kod (Etapy B/C), realny koszt fine-tuningu (API SFT lub własna infrastruktura dla DeepSeek), oraz ewaluacja na WIELU konfiguracjach testowych (nie tylko oryginalnym 3-ID mini-DBC – inaczej znów mierzymy zapamiętywanie, nie generalizację). To uzasadnia status osobnego eksperymentu, nie doklejki.

6. OTWARTE PYTANIA DO WYKŁADOWCY (PRZED STARTEM ETAPÓW B-D)

- 1 Czy akceptowalne, żeby "nauczycielem" był nasz klasyczny klasyfikator, czy wykładowcy zależało konkretnie na tym, żeby LLM SAM odkrywał wzorce?
- 2 Jaki zakres różnorodności korpusu (Etap A) uznać za wystarczający przed przejściem do faktycznego fine-tuningu?
- 3 Zaczynamy od GPT-4.1/o4-mini (łatwiejszy dostęp) czy DeepSeek (open-weight, pełna kontrola, własna infrastruktura)?
- 4 Czy warto najpierw sprawdzić tańszy wariant pośredni: setki przykładów w kontekście (long-context few-shot) zamiast pełnego fine-tuningu wag?

MagistralaCAN4 – Eksperyment_4.3_Propozycja_Bootstrapped_FineTuning_20260728.md,
esp_experiment_4_3/README.md